

DataFest Hackathon 2026

Challenge 1 — Energical AI Assistant

Agent IA de vente conversationnel pour le catalogue Energical

Rapport technique — Architecture finale d'équipe

Data Science Club USTHB × Energical × ACS Club

Juin 2026

1. Introduction et contexte

Energical est le premier site algérien de vente en ligne de produits d'électricité, de plomberie sanitaire et de menuiserie. Les clients arrivent souvent avec un besoin exprimé en langage naturel et peu technique (par exemple « je veux une porte ») alors que le catalogue compte 731 références. Ce rapport documente la solution finale apportée par l'équipe au Challenge 1 : un agent conversationnel basé sur un LLM, exposé via une API, et consommé par une interface web dédiée.

Ce document couvre l'approche LLM retenue, l'architecture technique complète (backend agent + exposition API + interface web), le processus d'itération qui a permis d'aboutir à une solution fiable, les résultats obtenus, ainsi que les limites actuelles et les pistes d'amélioration.

2. Approche LLM choisie

L'approche retenue est un agent conversationnel basé sur un modèle de langage local (LLM), combiné à une recherche augmentée par récupération (RAG) sur le catalogue produits. Ce choix répond directement à l'exigence du challenge : un agent capable de comprendre des formulations variées et imprévisibles, plutôt qu'une logique figée de type if/else, explicitement jugée insuffisante par le cahier des charges.

2.1 Sélection du modèle de langage

Le choix du modèle de langage local a nécessité plusieurs itérations :

- phi3:mini — premier modèle testé pour sa légèreté. Écarté car incompatible avec l'appel d'outils (tool calling) via Ollama, fonctionnalité indispensable à l'architecture RAG retenue.
- qwen2.5-coder:7b — second modèle testé. Compatible avec l'appel d'outils, mais comportement instable (format d'appel d'outil parfois mal respecté, déclenchement non systématique).
- llama3.1:8b — modèle finalement retenu pour le pipeline final, offrant un appel d'outils fiable et constant sur l'ensemble des scénarios testés.

Le modèle d'embeddings utilisé pour la recherche vectorielle est nomic-embed-text:latest, également exécuté localement via Ollama.

2.2 Pourquoi une approche RAG plutôt qu'un prompt unique

Le catalogue de 731 produits ne peut pas être transmis intégralement à chaque requête du modèle, pour deux raisons : la fenêtre de contexte limitée des modèles locaux légers, et l'exigence de minimisation des données du NDA liant Energical et le Data Science Club USTHB. L'approche RAG permet de ne transmettre au modèle, à chaque question, que les quelques produits les plus pertinents au regard de la demande du client — jamais le catalogue complet.

3. Architecture technique finale

L'architecture finale, telle que déployée par l'équipe, s'organise en trois couches : l'orchestration de l'agent IA (n8n), le moteur de langage local (Ollama), et une interface web qui consomme l'agent comme une API backend.

3.1 Composants

Composant	Rôle
n8n (Docker, local)	Orchestration du workflow : indexation du catalogue, agent conversationnel, appel d'outils, mémoire de conversation.
Webhook (n8n)	Point d'entrée HTTP du workflow, recevant les messages envoyés par l'interface web et déclenchant l'agent.
Trigger manuel + Schedule Trigger (n8n)	Déclenchement de l'indexation du catalogue, soit manuellement à la demande, soit automatiquement à intervalle régulier.
Ollama	Exécution locale du LLM (llama3.1:8b) et du modèle d'embeddings (nomic-embed-text).
Vector Store en mémoire (n8n)	Stockage des vecteurs du catalogue produits pour la recherche par similarité sémantique.
AI Agent + Simple Memory (n8n)	Agent conversationnel pilotant le LLM, avec accès à l'outil de recherche catalogue et mémoire des échanges précédents dans la session.
HTTP Request (n8n → interface web)	Renvoie la réponse de l'agent vers l'interface web développée par l'équipe, qui l'affiche au client.

3.2 Schéma du pipeline

Indexation (une fois) : lecture du fichier `energical_catalogue_produits.csv` → découpage en documents par le Data Loader → génération des embeddings (nomic-embed-text) → stockage dans le Vector Store en mémoire.

Conversation (à chaque message) : le client interagit avec l'interface web développée par l'équipe → la requête est transmise au Webhook n8n → l'AI Agent, guidé par son System Message et sa mémoire de conversation, interroge l'outil de recherche catalogue si la demande porte sur un produit → le Vector Store renvoie les produits les plus proches sémantiquement → le LLM formule une réponse en langage naturel → la réponse est renvoyée à l'interface web via une requête HTTP, pour affichage au client.

Cette séparation entre le backend agent (n8n) et le frontend (interface web) a permis un développement en parallèle au sein de l'équipe : la partie agent IA et la partie interface utilisateur ont été conçues indépendamment, connectées par un contrat d'API simple (Webhook en entrée, HTTP Request en sortie).

La branche d'indexation du catalogue peut être déclenchée de deux façons : manuellement (nœud « When clicking 'Execute workflow' », utile en développement et pour forcer une réindexation à la demande) ou automatiquement à intervalle régulier (nœud Schedule Trigger), afin de maintenir le Vector Store à jour sans intervention manuelle si le catalogue venait à être modifié.

3.3 Respect du NDA (Article 4)

- Aucune donnée Energical n'est envoyée à un service cloud externe : le LLM, les embeddings et le Vector Store tournent tous localement sur la machine de développement, via Docker.
- Le seul appel réseau du pipeline vers un moteur d'IA est local (<http://localhost:11434> pour Ollama).
- Les fichiers du pack de données sont stockés uniquement dans un volume Docker local, jamais commités sur GitHub (exclusion via .gitignore dès le début du projet).
- À chaque requête, le modèle ne reçoit que les quelques produits pertinents retournés par le Vector Store, jamais le fichier catalogue complet.
- L'usage d'un service vectoriel cloud externe (Pinecone notamment) a été envisagé puis écarté, faute de validation écrite explicite d'Energical pour l'envoi de données — même sous forme de vecteurs — vers un service tiers.

4. Processus d'itération et de fiabilisation

Au-delà du choix initial des composants, une part importante du travail a consisté à fiabiliser le comportement de l'agent, en particulier sur trois points critiques pour la confiance du client final : l'absence d'hallucination, le respect des règles métier d'Energical, et la cohérence linguistique.

4.1 Hallucination de produits inexistant

Un test de robustesse a consisté à interroger l'agent sur un produit fictif (une porte avec des caractéristiques inventées ne figurant pas dans le catalogue), combiné à un budget client élevé. Le comportement initial de l'agent était problématique :

« Avec un budget de 9999999 DA, vous avez accès à toutes nos portes vitrées en MDF Nid Abeille ! [...] disponible immédiatement [...] Elle coûte 10 000 DA [...] »

L'agent affirmait la disponibilité d'un produit inexistant, avec un prix et des caractéristiques entièrement inventés. La correction a consisté à restructurer le System Message selon un format Rôle / Outils / Règles / Instructions / Exemples / Rappel final, avec des règles anti-hallucination explicites interdisant d'inventer un produit, un prix ou une disponibilité, et obligeant l'agent à répondre honnêtement en l'absence de correspondance exacte.

Une difficulté technique additionnelle est apparue lors de cette correction : un exemple du prompt utilisait des espaces réservés entre crochets (« [prix réel en DA] ») pour illustrer le format attendu. Le modèle reproduisait ces crochets littéralement au lieu de les remplacer. La correction a consisté à utiliser des valeurs concrètes dans les exemples plutôt que des gabarits, pour que le modèle apprenne une substitution complète.

Après correction, sur le même scénario, l'agent reconnaît l'absence du produit demandé et propose une alternative réelle du catalogue, sans affirmer de correspondance exacte ni inventer de caractéristiques.

4.2 Exposition technique de l'outil et devise

Deux comportements indésirables additionnels ont été identifiés et corrigés au cours du développement : l'agent mentionnait initialement le nom technique de l'outil de recherche (Simple_Vector_Store) directement dans sa réponse au client, et exprimait un prix en euros plutôt qu'en dinars algériens. Ces deux points ont été corrigés par l'ajout de règles explicites dans le System Message.

4.3 Cohérence linguistique

Le cahier des charges demande que l'agent guide le client « dans la langue de son choix ». Une règle a été ajoutée demandant à l'agent de répondre dans la même langue que le message du client. Un test en anglais a cependant révélé que cette règle, initialement placée au milieu de la liste des autres règles métier, n'était pas systématiquement respectée : une question posée en anglais (« hello do u have doors ») a reçu une réponse en français.

Ce résultat suggère que la position et la priorité d'une instruction dans le prompt influencent significativement son respect par le modèle, en particulier lorsque les exemples illustratifs du prompt sont eux-mêmes rédigés dans une seule langue (français). La correction proposée consiste à placer cette règle en tête du System Message, comme règle absolue prioritaire, et à préciser explicitement que la langue des exemples ne doit pas influencer la langue de réponse. Cette correction a été identifiée mais sa validation systématique sur plusieurs langues (anglais, arabe, darija) reste à finaliser.

5. Résultats obtenus

Le développement s'est concentré sur une famille de produits traitée de façon robuste plutôt que sur une couverture large mais superficielle des neuf familles disponibles, conformément à la recommandation du cahier des charges (« 1 famille produit bien traitée > 3 bâclées »).

5.1 Famille traitée : Portes

- Recherche d'un produit précis du catalogue : l'agent retrouve correctement la référence demandée par le client.
- Détection d'une rupture de stock : l'agent indique clairement l'indisponibilité plutôt que d'affirmer une disponibilité erronée.
- Proposition d'une alternative cohérente : l'alternative retournée par la recherche sémantique correspond exactement au produit_alternatif_id défini officiellement par Energical dans le catalogue, sur le cas testé.
- Robustesse face à une demande inexistante : un produit aux caractéristiques inventées n'est plus signalé comme disponible, après correction du comportement initial d'hallucination.
- Dialogue progressif et mémoire de conversation : l'agent pose des questions de clarification avant de recommander un produit, et conserve le contexte des échanges précédents grâce à un module de mémoire de session (memoryBufferWindow).

- Respect de la devise : les réponses expriment les prix en dinars algériens (DA).
- Absence de jargon technique : les réponses ne mentionnent plus le nom interne de l'outil de recherche.

5.2 Intégration d'équipe

L'agent est exposé via un Webhook n8n, consommé par une interface web développée en parallèle par un autre membre de l'équipe. Cette architecture a permis de découpler le développement du moteur conversationnel de celui de l'interface utilisateur, chaque partie pouvant être testée indépendamment avant intégration.

6. Limites et pistes d'amélioration

Conformément à la recommandation du jury de privilégier l'honnêteté sur les limites plutôt que de défendre une solution qui ne fonctionne pas, les points suivants n'ont pas été pleinement validés dans le temps imparti :

- Couverture limitée à une seule famille de produits (Portes) sur les neuf familles disponibles dans le fichier `energical_arbres_decision.json`.
- Les règles métier détaillées de l'arbre de décision n'ont pas été intégrées comme outil dédié de l'agent ; le comportement de proposition d'alternative repose sur la similarité sémantique du Vector Store, fiable sur les cas testés mais non validée systématiquement.
- Le respect de la consigne « répondre dans la langue du client » a été identifié comme imparfait (réponse en français à une question en anglais) ; une correction a été proposée (repositionnement de la règle en tête de prompt) mais n'a pas encore été revalidée sur un panel de langues, notamment le darija.
- Le Vector Store utilisé est un stockage en mémoire propre à l'exécution n8n : il ne persiste pas après un redémarrage et nécessiterait une solution comme Qdrant pour un déploiement en production.
- Les tests effectués sont manuels et ponctuels ; aucune métrique quantitative (taux de réussite, temps de réponse moyen, taux d'hallucination résiduel) n'a été établie sur un échantillon de questions.

Avec un temps de développement supplémentaire, les priorités seraient : revalider la cohérence linguistique sur plusieurs langues dont le darija, intégrer explicitement les neuf arbres de décision métier comme outil dédié, migrer vers un Vector Store persistant (Qdrant), et constituer un jeu de questions de test pour mesurer objectivement la fiabilité des recommandations et le taux d'hallucination résiduel.

7. Conclusion

La solution finale de l'équipe répond aux critères minimaux du Challenge 1 : un véritable LLM doté d'un raisonnement par appel d'outils opère sur les données réelles du catalogue Energical, détecte les ruptures de stock, propose des alternatives cohérentes, et engage un dialogue à plusieurs tours avant de formuler une recommandation. L'agent est exposé via une API (Webhook n8n) consommée par une interface web dédiée, permettant un développement en parallèle au sein de l'équipe. Le processus de développement a mis en évidence et corrigé plusieurs risques significatifs (hallucination, exposition de détails techniques, devise incorrecte),

et a identifié une limite restant à corriger sur la cohérence linguistique. L'ensemble du pipeline respecte les exigences de confidentialité du NDA en exécutant le moteur d'IA entièrement en local.